Relatório – Exercício 3: Multiplicação de inteiros com SFOTS

João Carlos Simões Silva · a110901 · Lógica Computacional

1. Introdução

Neste trabalho tínhamos de modelar um algoritmo de multiplicação de inteiros como um SFOTS em pySMT e depois provar umas propriedades com k-indução, usando o Z3 como solver. O algoritmo é este:

```
x, y, z = a, b, 0
while y != 0:
    if y é par:  x = x << 1;  y = y >> 1
    senão:       z = z + x;   y = y - 1
```

No fim, `z = a*b`. Usámos bitvectors de `n = 8` bits, sem sinal.

2. Alínea a) – O modelo SFOTS

O estado é `(pc, a, b, x, y, z)`. O `pc` diz em que nodo do CFA estamos (`init`, `skip`, `left`, `right`, `stop`, `error`) e usa 3 bits. Os valores `a` e `b` nunca mudam, mas precisamos deles para o invariante.

Algumas decisões tomadas::

- A aritmética dos bitvectors é módulo 2^n e nunca dá erro por si só, por isso o **overflow** tem de ser detetado nas transições. Em `x << 1` há overflow quando `x >= 2^(n-1)`. Na soma `z + x` há overflow quando o resultado (módulo 2^n) é menor do que `z`.
- Cada ramo do CFA é uma cláusula da relação de transição, e a transição de `left` e de `right` tem duas versões: uma normal (sem overflow) e uma que vai para `error` (com overflow). As condições são complementares, por isso o erro tem sempre prioridade.
- O estado inicial já inclui a atribuição `x, y, z = a, b, 0`, para o invariante valer logo desde o início.
- `stop` e `error` têm um ciclo sobre si próprios, para a relação de transição ser total.

3. A função de k-indução

Escrevemos uma função `k_induction(k, init, declare, trans, prop)` que faz as duas partes da prova:

- **Caso base:** vê se a propriedade vale nos primeiros `k` estados a partir do estado inicial.
- **Passo indutivo:** assume que vale em `k` estados seguidos e vê se vale no seguinte (sem impor o estado inicial).

Em ambos pedimos ao Z3 um contra-exemplo. Se não existir (*unsat*), a parte está provada. Um falhanço do passo indutivo nem sempre significa que a propriedade é falsa: pode ser só um estado inalcançável a estragar a prova.

4. Alínea b) – O invariante x*y + z = a*b

Provámos o invariante `x*y + z = a*b` escrito diretamente com a aritmética dos bitvectors.

Com `k = 1` o passo indutivo falha, porque o invariante sozinho não diz que `y` é par quando estamos em `left` (e a transição `left` assume isso). Com `k = 2` já funciona, porque olhando dois passos para trás vemos a transição `skip → left`, que só acontece com `y` par. Deixámos o 14, que também é uma prova válida.

5. Alínea c) – O erro é inalcançável

Nesta alínea acrescentámos ao estado inicial `a < 2^(n/2)` e `b < 2^(n/2)` (com `n = 8`, isto dá `a, b < 16`, logo `a*b < 256`) e queríamos provar que nunca se chega a `error`.

O problema é que provar só `Not(error)` **não funciona**, nem com `k` grande: o Z3 arranja sempre estados inalcançáveis, sem relação nenhuma entre `x, y, z` e `a, b`, de onde se chega ao erro. Tivemos de reforçar a propriedade com:

- as restrições `a, b < 2^(n/2)` (mantêm-se porque `a` e `b` não mudam);
- o invariante `x*y + z = a*b`, mas desta vez **em 2n bits** (estendendo as variáveis com zeros). Assim o produto nunca dá a volta e o invariante fala dos valores inteiros verdadeiros, o que garante `x*y + z = a*b < 256` e impede os overflows.

Com isto a propriedade prova-se com `k = 2`. Como implica `Not(error)`, o erro fica inalcançável.

6. Alínea d) – Animação

Entendemos a animação como duas coisas: o sistema **nunca bloqueia**, e a execução **termina**.

Invariante auxiliar. Para o argumento da terminação precisamos de saber que `y != 0` em `left` e `right`, e que `y` é par em `left e ímpar em right`. Chamámos-lhe `inv_d`. Provámo-lo por k-indução com `k = 1`.

Variante. Definimos, em `n + 2` bits (para `2y + 2` não dar a volta):

- `V = 0` se `pc` é `stop` ou `error`;
- `V = 2*y + c(pc)` nos outros casos, com `c(init) = 2`, `c(skip) = 1`, `c(left) = c(right) = 0`.

O Z3 confirma que, a partir de qualquer estado que satisfaça `inv_d` e não seja terminal, `V` diminui em todas as transições . Como `V` toma valores naturais, não pode haver execuções infinitas.

Sem bloqueios. Pedimos ao Z3 um estado com `pc <= 5` sem nenhum sucessor, e não existe. Cada nodo tem sempre uma transição aplicável.

Juntando as duas coisas, qualquer execução chega a `stop` ou `error`. Com as hipóteses da alínea c), o `error` é inalcançável, logo a execução acaba em `stop`.

7. Uso de LLM’s

O uso de LLM´s neste trabalho serviu para nos explicar o que era pedido em cada alínea e para esclarecer conceitos como a k-indução, o variante e a propriedade de animação. Também o usámos para organizar as ideias e a estrutura do relatório assim como as cédulas de texto no notebook.